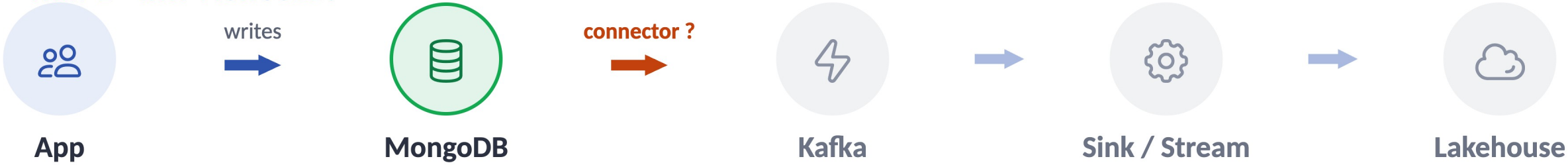
MongoDB — The Source Side of the Pipeline

Today's goals — by the end of this lesson you can:

1. Place MongoDB in a data platform
2. Explore a collection and spot schema problems
3. Query and aggregate, thinking in SQL
4. Move changes out with Change Streams + Kafka

Where MongoDB Sits



mobile / web | **TODAY** | topics | consumers | Snowflake / Iceberg

Operational world | **Analytical world (Kafka lessons)**

- Recap: producers → topics → consumers / sink connectors
- Today we zoom into the **left edge**: the operational database
- MongoDB is the system of record for the **app**, not for analytics
- Our job: get changes **out**, reliably, without hurting the app

Document Model vs Tables

PART 1 · WHY MONGODB?

SQL: one order = 3 tables + 2 JOINs

orders	customer_id	total	status
ORD-10000	C0008	499.00	paid

order_items	sku	qty	unit_price
ORD-10000	SKU-104	1	499.0

customers	name	email
C0008	Noa Avraham	noa@example.com

=

MongoDB: one document

```
{
  "_id": ObjectId("6a88a594…"),
  "order_id": "ORD-10000",
  "customer": {
    "customer_id": "C0008",
    "name": "Noa Avraham",
    "contact": { "email": "noa@example.com" } },
  "items": [
    { "sku": "SKU-104", "qty": 1, "unit_price": 499.0 } ],
  "total": NumberDecimal("499.00"),
  "status": "paid"
}
```

Schema on Write vs Schema on Read

PART 1 · WHY MONGODB?

	Schema on write	Schema on read
Defined	In the database (CREATE TABLE)	In the code that reads it
Checked	On every INSERT	When someone reads the data
Bad data	Rejected with an error	Accepted, stored silently
New shape	ALTER TABLE / migration	Just write the new shape
Examples	PostgreSQL, Snowflake Silver	MongoDB, JSON files, Bronze

Bronze = schema on read: keep everything as it came. **Silver** = schema on write: enforce names and types. MongoDB does not enforce for you, so **your pipeline must**.

When MongoDB Shines

PART 1 · WHY MONGODB?

Whole entities

Order, profile, device state — read and written as one

Fast-changing shape

New fields every app release, no migration first

Polymorphic records

Different product types, different fields

Scale-out writes

High write volume with sharding

When NOT to Use MongoDB

Analytics over the whole dataset (big scans, GROUP BYs)	→ **Warehouse**
Highly relational data, many-to-many JOINs	→ **PostgreSQL**
Full-text and log search at scale	→ **Elasticsearch**
Pure key lookups, caching, microsecond latency	→ **Redis**

MongoDB vs the Neighbours

PART 1 · WHY MONGODB?

	Data model	Consistency (default)	Query power	Scaling	Best fit
MongoDB	Documents (BSON)	w:"majority" writes by default (5.0+)	Filters, aggregation, indexes	Replica sets + native sharding	App data, evolving JSON
PostgreSQL	Relational tables	ACID; sync or async replicas	Full SQL + JOINs	Vertical + read replicas	Relational OLTP
Cassandra	Wide-column	Tunable per query (driver default ONE)	CQL by partition key	Masterless, add nodes	Write-heavy time series
DynamoDB	Key-value / document	Eventual reads by default; strong opt-in	Key + index access	Managed partitioning	Serverless on AWS
Redis	In-memory key-value	Async replication by default	Key operations	Cluster mode	Cache, sessions
Elasticsearch	Inverted index	Near real-time (~1 s refresh)	Full-text + aggregations	Shards + replicas	Search, logs (ELK)

Real Use Cases: Toyota Connected

PART 1 · THE OTHER SIDE OF NETFLIX & UBER

- **Toyota Connected** builds the platform behind **Toyota Safety Connect**: crash notification and emergency help
- **Problem:** the legacy database had reliability issues, and a crash alert cannot wait for a fix
- **Why MongoDB:** target of **99.99% availability**, upgrades without downtime (multi-region), developers create collections without a DBA
- **Architecture:** AWS microservices over **20+ MongoDB Atlas databases**, 9M+ vehicles, data reaches safety agents in ~3 seconds



Not chosen: staying on the legacy database. These are company and vendor case studies: marketing with real facts inside.

Real Use Cases: Forbes

PART 1 · THE OTHER SIDE OF NETFLIX & UBER

- **Forbes** moved its digital platform to **MongoDB Atlas on Google Cloud** (migration took ~6 months, done late 2019)
- **Problem:** a self-managed, on-prem setup slowed releases; provisioning and maintenance were a burden
- **Why MongoDB:** managed service, easy replication and provisioning, flexible documents behind a stable API
- **Reported results:** 58% faster builds, 2×–10× faster release cycles, 25% lower total cost

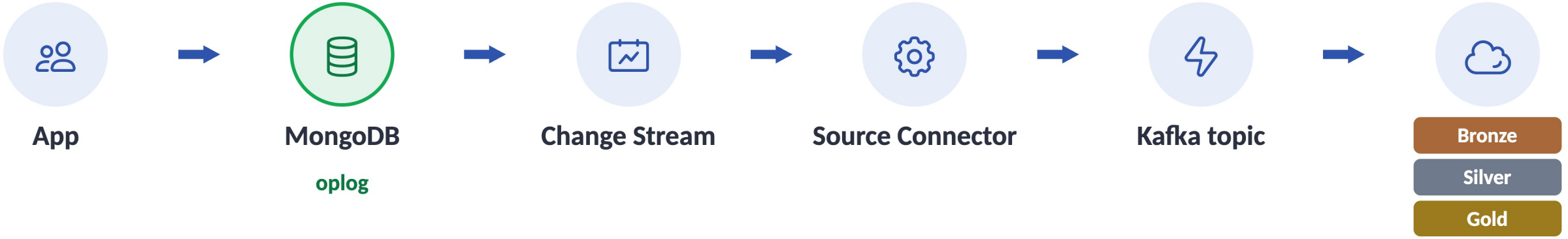
FORBES

Not chosen: keeping self-managed on-prem MongoDB. Next step: replace Solr with Atlas Search. Results are company-reported.

The Modern Architecture

PART 1 · APP → MONGODB → CDC → KAFKA → LAKEHOUSE

OUTDATED Then: **Hadoop / Storm** → daily bulk load → **MongoDB** (eBay, 2012)



- **Now:** MongoDB is the **source**; every change streams out (CDC)
- Resume token kept in Kafka Connect offsets — no nightly dumps, no load on the primary

BSON and Data Types

PART 2 · DATA MODELING

BSON type	What breaks downstream
ObjectId	Arrives as {"$oid": "..."}; first 4 bytes = creation time
Decimal128	Arrives as {"$numberDecimal": "499.00"} — a string inside an object
Date	Arrives as {"$date": ...}; UTC millis — timezone assumptions break
Int32 / Int64 / Double	Same field, three numeric types → Spark infers string or fails

```
// 4 docs, 4 types, 1 field
db.types_demo.insertMany([
  { price: 499 },
  { price: 499.0 },
  { price: NumberDecimal("499.00") },
  { price: "499.00" } ]);


db.types_demo.aggregate([{ $project:
  { _id: 0, price: 1,
    t: { $type: "$price" } } }]);

// → int, double, decimal, string
```

⚠ **In SQL a column has one type. In MongoDB a field can have four — the #1 bug we hunt today.**

Collections and Documents

PART 2 · DATA MODELING

- Collection = group of documents, **no enforced columns**
- Every document has a unique **_id** — your downstream MERGE key
- Max document size **16 MB**; nesting and arrays allowed
- Created **implicitly** on first insert — typos create collections

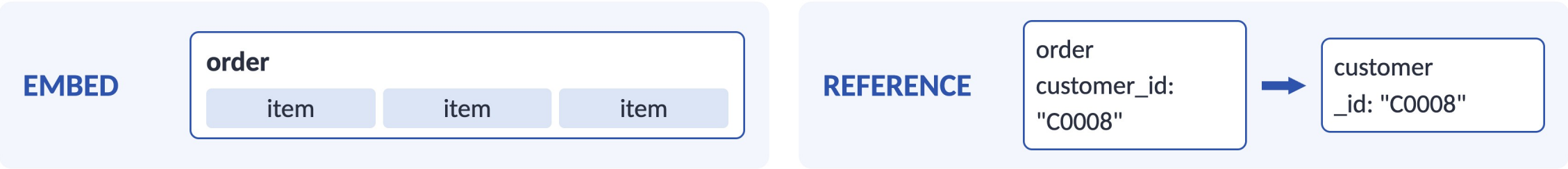
```
use shop

// typo → a NEW collection
db.oders.insertOne({ x: 1 })
show collections

// orders, oders
db.oders.drop()  // clean up
```

Embedding vs Referencing

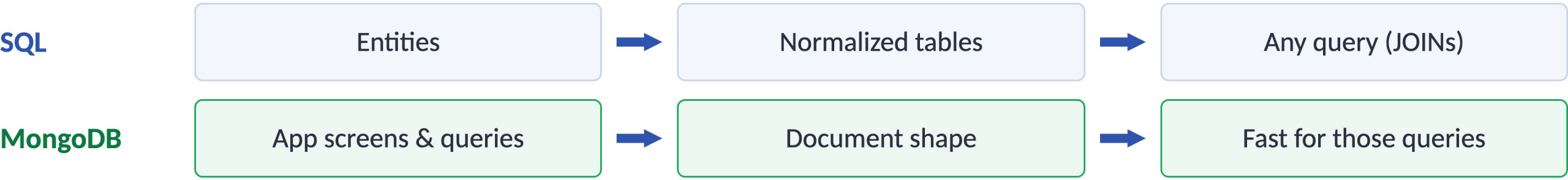
PART 2 · DATA MODELING



	Embed	Reference
SQL analogy	Pre-joined, denormalized row	Foreign key (not enforced!)
Read / update	One fast read; duplicates to update	$lookup or 2 queries; update once
Growth	Unbounded arrays hit 16 MB	No size issue
Pipeline impact	Arrays must be exploded; one change = whole parent	Clean 1:1 tables; joins move to the warehouse

Modeling by Access Patterns

PART 2 · DATA MODELING



- The schema reflects **the app's screens**, not business entities
- Expect duplication (customer name in every order) and pre-computed fields
- DE consequence: decide which copy is the "truth" in your Silver layer

Schema Drift and Its Cost

PART 2 · DATA MODELING · FLEXIBLE SCHEMA

App v1 · before 1 Jun

`customer.email`

107 orders

→

App v2 · from 1 Jun

`customer.contact.email`

93 orders

→

Silver table

`email = NULL`

for every new order

- No ALTER TABLE ⟶ releases just write new shapes
- Symptoms: NULL spikes, new columns, type conflicts, failed loads
- Defend: check $type / $exists, track schema_v, agree contracts

```
// how many orders per shape?
db.orders.aggregate([
  { $group: { _id: "$schema_v",
    n: { $sum: 1 } } } ]);

// → { _id: 1, n: 107 }, { _id: 2, n: 93 }
```

Schema Validation

PART 2 · DATA MODELING · $JSONSCHEMA

```
db.runCommand({ collMod: "orders",
 validator: { $jsonSchema: {
  bsonType: "object",
  required: ["order_id", "status",
             "created_at", "total"],
  properties: {
   order_id: { bsonType: "string" },
   status: { enum: ["created", "paid",
    "shipped", "delivered", "cancelled"] },
   created_at: { bsonType: "date" },
   total: { bsonType: "decimal" } } } },
 validationLevel: "moderate",
 validationAction: "error" });
```

```
db.orders.insertOne({
 order_id: "ORD-X",
 status: "SHIPPED",
 created_at: new Date(),
 total: "10" });

// MongoServerError:
// Document failed validation
```

- Like CREATE TABLE constraints — but **opt-in**
- Applies to **new writes**; old docs stay messy
- Start with "warn", then switch to "error"

Flattening Arrays for Analytics

PART 2 · DATA MODELING · OPTIONAL

order ORD-10003

- items[0] SKU-104 · qty 1
- items[1] SKU-101 · qty "2"
- items[2] SKU-102 · qty 3

explode →

order_id	item_idx	sku	qty
ORD-10003	0	SKU-104	1
ORD-10003	1	SKU-101	2
ORD-10003	2	SKU-102	3

- Nested object → dot columns: shipping.address.city
- Array of objects → child table, key = _id + index
- Use explode_**outer**: 40 orders have empty items

```
# PySpark (Silver layer)
items = orders_df.select("order_id",
 F.posexplode_outer("items")
  .alias("item_idx", "item"))
```

Insert and Find

PART 3 · CRUD · MONGOSH ON MONGODB 8.0

```
// docker exec -it mongo mongosh
use shop



// INSERT INTO orders VALUES (...)
db.orders.insertOne({
  order_id: "ORD-99999",
  customer: { customer_id: "C0999", name: "Test Student" },
  items: [{ sku: "SKU-101", name: "Yoga Mat", qty: 1, unit_price: 89.9 }],
  total: NumberDecimal("89.90"), status: "created", created_at: new Date()
});  // → { acknowledged: true, insertedId: ObjectId("…") }



// SELECT * FROM orders LIMIT 1
db.orders.findOne();

// SELECT * FROM orders WHERE order_id = 'ORD-99999'
db.orders.find({ order_id: "ORD-99999" });
```

Filter, Project, Sort, Limit

PART 3 · CRUD

```
// SELECT order_id, total, status FROM orders

// WHERE city = 'Haifa' AND status IN ('shipped','delivered') ORDER BY created_at DESC LIMIT 5
db.orders.find(
  { "shipping.address.city": "Haifa", status: { $in: ["shipped", "delivered"] } },  // WHERE
  { _id: 0, order_id: 1, total: 1, status: 1 }  // SELECT
).sort({ created_at: -1 }).limit(5);  // ORDER BY, LIMIT



// operators: $eq $ne $gt $gte $lt $lte $in $nin $exists $type $regex
```

```
db.orders.countDocuments(
  { total: { $gte: 1000 } })
// → 75
```

```
db.orders.countDocuments(
  { total: { $type: "string" } })
// → 19 — silently excluded above!
```

Update Operators and Upsert

PART 3 · CRUD

```
// UPDATE orders SET status = 'paid', updated_at = NOW() WHERE order_id = 'ORD-99999'
db.orders.updateOne({ order_id: "ORD-99999" },
  { $set: { status: "paid" }, $currentDate: { updated_at: true } });

// also: $inc $unset $push $pull $addToSet $rename



// UPDATE orders SET status = LOWER(status) — pipeline update
db.orders.updateMany({ status: { $exists: true } },
  [{ $set: { status: { $toLower: "$status" } } }]);  // modifiedCount: 30



// INSERT ... ON CONFLICT (order_id) DO UPDATE — upsert
db.orders.updateOne({ order_id: "ORD-77777" },
  { $set: { status: "created" }, $setOnInsert: { created_at: new Date() } },
  { upsert: true });
```

⚠ **Every update becomes a change event — by default carrying only the changed fields.**

Delete

PART 3 · CRUD

```
// DELETE FROM orders WHERE order_id = 'ORD-77777'
db.orders.deleteOne({ order_id: "ORD-77777" });

// DELETE FROM orders WHERE order_id IN (...)
db.orders.deleteMany({ order_id: { $in: ["ORD-99999", "ORD-77777"] } });

// TRUNCATE — no “are you sure?”    // DROP TABLE
db.orders.deleteMany({});                db.orders.drop();
```

⚠ Hard delete: the document is gone — only the **oplog** remembers

✓ Soft delete: $set deleted_at — a normal update downstream

Exploring an Unknown Collection

PART 3 · YOUR FIRST FIVE MINUTES

```
use shop
db.getCollectionNames();  // SHOW TABLES
db.runCommand({ listCollections: 1, nameOnly: true });


db.orders.estimatedDocumentCount();  // fast, from metadata
db.orders.countDocuments({});  // exact → 200
db.orders.distinct("status");  // SELECT DISTINCT → 14 spellings of 5 values!
db.orders.aggregate([{ $sample: { size: 3 } }]);  // random docs, not the oldest

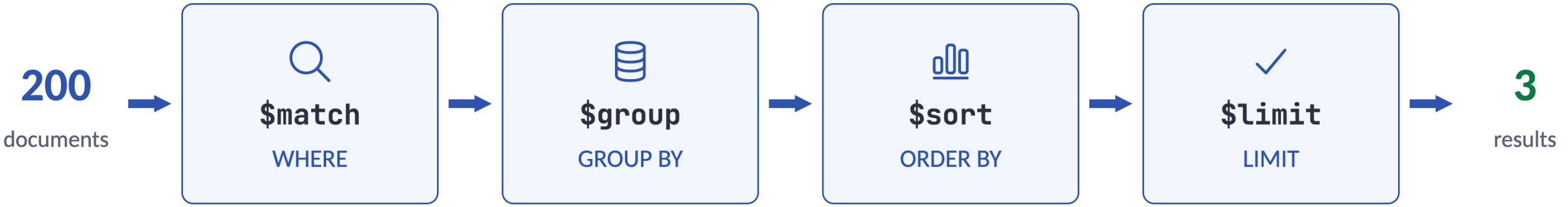


// how big is it? — check before a full export
db.orders.aggregate([{ $collStats: { storageStats: {} } }]);
db.orders.getIndexes();
```

Use $sample, not limit(): drift usually lives in the newest documents.

What Is a Pipeline

PART 4 · AGGREGATION BASICS



- Documents flow through **stages**; each output feeds the next
- You choose the order: **$match early** = less work later
- Same idea as Spark: filter → groupBy → agg → orderBy

$match, $group, $project vs SQL

PART 4 · REVENUE PER CITY, EXCLUDING CANCELLED

SQL

```
SELECT LOWER(REPLACE(
  city, '-', ' ')) AS city,
 COUNT(*) AS orders,
 SUM(CAST(total
  AS DECIMAL)) AS revenue
FROM orders
WHERE status <> 'cancelled'
GROUP BY 1
ORDER BY revenue DESC
LIMIT 3;
```

MongoDB

```
db.orders.aggregate([
 { $match: { status: { $ne: "cancelled" } } },  // WHERE
 { $group: {  // GROUP BY
   _id: { $replaceAll: { find: "-", replacement: " ",
     input: { $toLower: "$shipping.address.city" } } },
   orders: { $sum: 1 },  // COUNT(*)
   revenue: { $sum: { $convert: { input: "$total",
     to: "decimal", onError: null, onNull: null } } } } },
 { $sort: { revenue: -1 } }, { $limit: 3 },
 { $project: { _id: 0, city: "$_id", orders: 1, revenue: 1 } } ]);
```

→ netanya 28 · 31354.60 | haifa 26 · 29292.90 | jerusalem 18 · 24802.70 (after slide 20's lowercase fix)

$unwind and $lookup

PART 4 · EXPLODE ARRAYS, JOIN COLLECTIONS

```
// Top SKUs by quantity — like Spark explode() + GROUP BY
db.orders.aggregate([
  { $unwind: "$items" },  // one doc per array element (empty arrays dropped!)
  { $group: { _id: "$items.sku", name: { $first: "$items.name" },
      qty: { $sum: { $toInt: "$items.qty" } } } },  // qty is sometimes "2"
  { $sort: { qty: -1 } }, { $limit: 3 } ]);

// → SKU-105 Backpack 100 · SKU-106 Sunglasses 98 · SKU-107 Bike Helmet 92



// LEFT JOIN customers c ON o.customer.customer_id = c._id
db.orders.aggregate([
  { $lookup: { from: "customers", localField: "customer.customer_id",
      foreignField: "_id", as: "customer_doc" } }, { $limit: 1 } ]);
```

⚠ **Heavy JOINs belong in the warehouse, not on the app's database.**

Data Quality Checks

PART 4 · OPTIONAL · A MINI DQ REPORT IN 3 QUERIES

```
// 1. Type profile — impossible in SQL (a column has one type)
db.orders.aggregate([{ $group: { _id: { $type: "$total" }, n: { $sum: 1 } } }]);

// → decimal 115 · double 55 · string 19 · missing 11
```

```
// 2. Duplicate business keys — GROUP BY order_id HAVING COUNT(*) > 1
db.orders.aggregate([{ $group: { _id: "$order_id", n: { $sum: 1 } } },
  { $match: { n: { $gt: 1 } } }]);  // → ORD-10010 (2), ORD-10011 (2)
```

```
// 3. Missing required fields — WHERE status IS NULL
db.orders.countDocuments({ status: { $exists: false } });  // → 7
```

Why Indexes Matter

PART 5 · INDEXES

COLLSCAN

No index: read **every** document

200 read to return 11

IXSCAN

B-tree: jump straight to the keys

11 read to return 11

- Indexes cost RAM and slow every write
- They are built for the **app's** queries, not your export
- DE angle: an unindexed extraction query can hurt production

Single and Compound Indexes

PART 5 · INDEXES

```
// CREATE INDEX ix_customer ON orders(customer_id)
db.orders.createIndex({ "customer.customer_id": 1 });



// Compound: Equality → Sort → Range (the ESR rule)
db.orders.createIndex({ status: 1, created_at: -1 });

// OK: find({ status: "paid" }).sort({ created_at: -1 })

// OK: find({ status: "paid" })    — uses the prefix

// NO: find().sort({ created_at: -1 })  — no "status" prefix



// Multikey: an index on an array field indexes every element
db.orders.createIndex({ "items.sku": 1 });
db.orders.getIndexes();
```

Same left-prefix rule as a SQL composite index.

Reading explain()

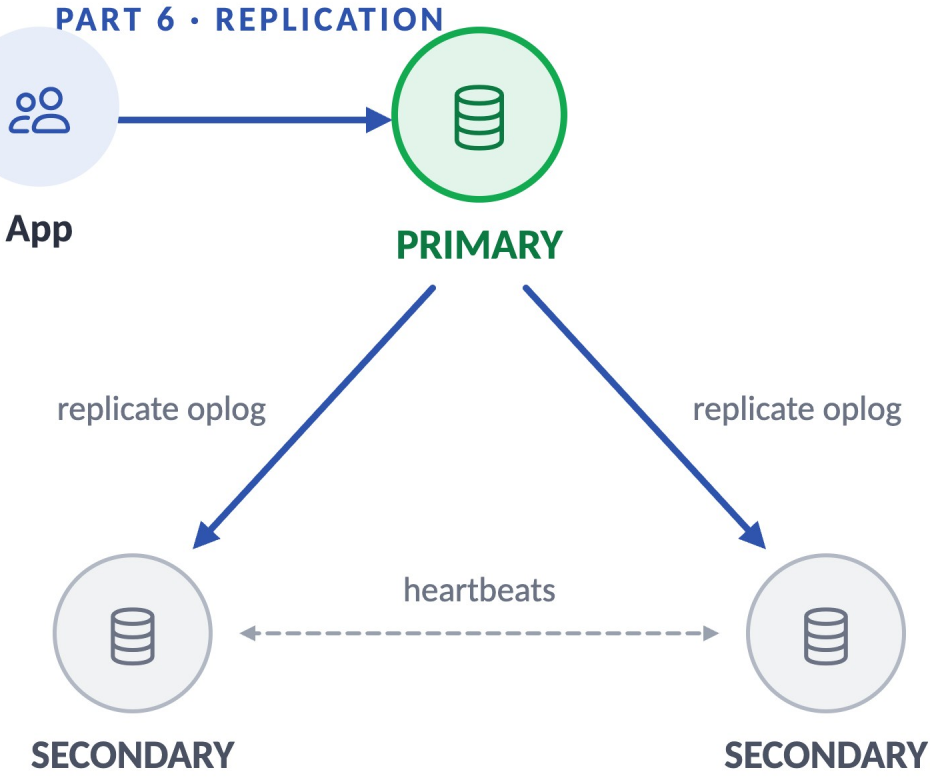

```
db.orders.dropIndex({ "customer.customer_id": 1 });  // start without it
db.orders.find({ "customer.customer_id": "C0008" }).explain("executionStats");


db.orders.createIndex({ "customer.customer_id": 1 });
db.orders.find({ "customer.customer_id": "C0008" }).explain("executionStats");
```

	stage	nReturned	keys examined	docs examined
Before index	COLLSCAN	11	0	200
After index	FETCH ← IXSCAN	11	11	11

Healthy query: docs examined ≈ nReturned

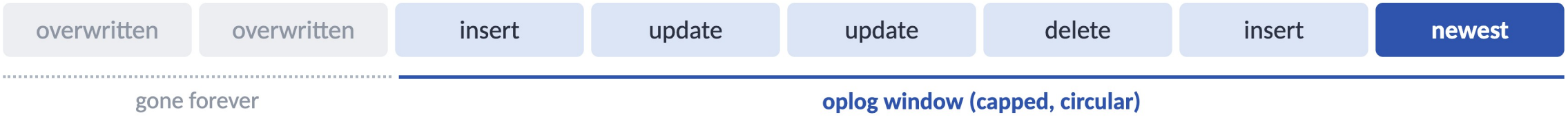
Replica Sets and Elections



- 1 **primary** takes all writes; **secondaries** copy it
- Primary down → members **elect** a new one in seconds
- Typical: 3 members (odd → a majority is possible)
- Our lab: a **1-member** replica set — enough for the oplog and change streams

The Oplog: Foundation of CDC

PART 6 · REPLICATION



- local.oplog.rs: every write on the primary, in order
- **Capped**: oldest entries overwritten
- Idempotent: replay twice = same result
- Use Change Streams, not raw oplog

```
use local
db.oplog.rs.find({ ns: "shop.orders" })
  .sort({ $natural: -1 }).limit(1);

// { op: "u", ns: "shop.orders",
//   o: { $v: 2, diff: { u: { status: "paid" } } },
//   o2: { _id: ObjectId("…") }, ts: Timestamp(…) }
rs.printReplicationInfo();  // your window
```

Read and Write Concerns

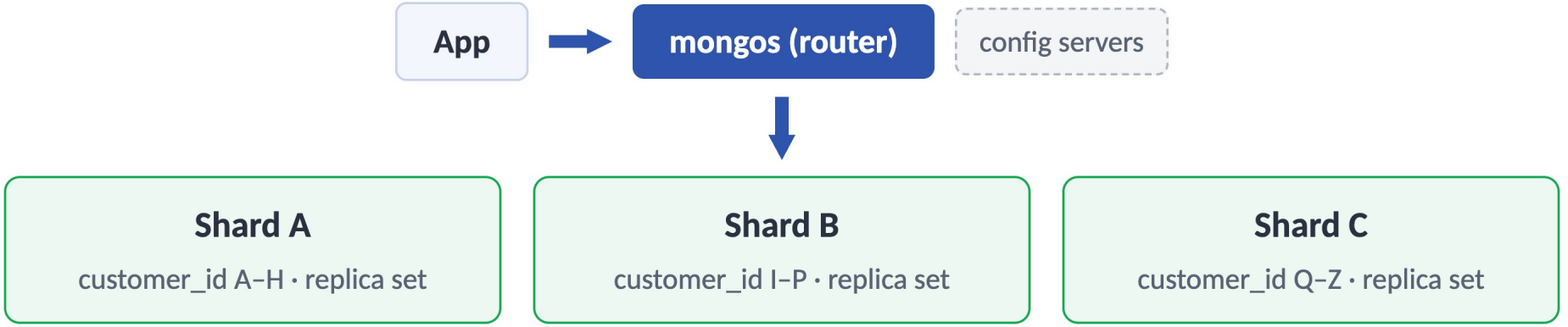
PART 6 · HOW SURE DO YOU NEED TO BE?

Setting	Meaning	Trade-off
w: 1	Primary acknowledged	Fast; can roll back on failover
w: "majority"	Majority has it — default since 5.0	Durable; slightly slower
readConcern "local"	Latest data on this node	May later be rolled back
readConcern "majority"	Only majority-committed data	Never rolled back — what CDC wants

```
db.orders.insertOne({ order_id: "ORD-W1" }, { writeConcern: { w: "majority", wtimeout: 5000 } });
db.orders.find({}).readConcern("majority").limit(1);
```

Sharding (Concept Only)

PART 6 · OPTIONAL · HORIZONTAL SCALING



- Bad shard key → hot shard; query without the key → hits every shard
- Same idea as Kafka partitions keyed by message key
- For CDC: change streams merge events from all shards for you

Transactions

PART 6 · BEGIN … COMMIT · PYMONGO 4.18+

```
# requires a REPLICA SET (or sharded cluster) — not a standalone mongod
from pymongo import MongoClient
client = MongoClient("mongodb://mongo:27017/?directConnection=true")
db = client.shop

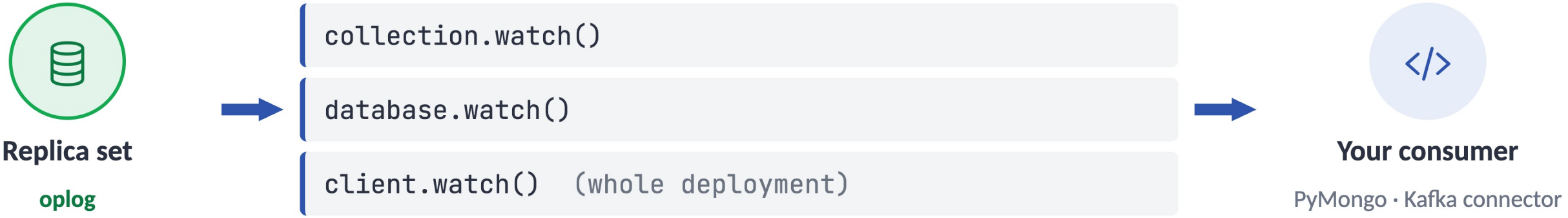

def place_order(session):
    db.orders.insert_one({"order_id": "ORD-TX1", "status": "paid"}, session=session)
    db.inventory.update_one({"sku": "SKU-101"}, {"$inc": {"stock": -1}},
                            upsert=True, session=session)


with client.start_session() as s:
    s.with_transaction(place_order)  # both or neither; retries transient errors
```

Single-document writes were always atomic · each write still arrives as its own change event

What Are Change Streams

PART 7 · CHANGE STREAMS · MONGODB'S BUILT-IN CDC



- A never-ending cursor: one event per change, **in order**
- Stable, documented format — never parse the raw oplog
- Only **majority-committed** changes; resumable
- Needs a replica set or sharded cluster — not standalone

Event Types and fullDocument

PART 7 · CHANGE STREAMS

```
// UPDATE event — default = deltas only
{ _id: { _data: "8268…" },  // resume token
  operationType: "update",
  clusterTime: Timestamp({ t: 1761310000, i: 1 }),
  ns: { db: "shop", coll: "orders" },
  documentKey: { _id: ObjectId("6a88…") },
  updateDescription: {
    updatedFields: { status: "paid" },
    removedFields: [] } }
```

operationType	fullDocument?
insert, replace	Always
update	Only with updateLookup / whenAvailable / required
delete	Never (pre-images only if enabled)

⚠ **updateLookup** fetches the *current* document at send time — it may already be newer, or null if deleted.

Resume Tokens

PART 7 · CHANGE STREAMS · FAULT TOLERANCE



Crash → resume

Saved token = 7 → restart with **resume_after** → continue at 8

At-least-once: event 7 may repeat → MERGE by _id

Token too old

Bookmark in slots 1–3 (overwritten) → **ChangeStreamHistoryLost**

Only fix: a new full snapshot

Save the token after processing — same rule as Kafka consumer offsets.

Live Demo: Watching a Collection

PART 7 · WATCH_ORDERS.PY · PYMONGO 4.18+ · MONGODB 7.0+

```
from bson import json_util
from pymongo import MongoClient
from pathlib import Path


coll = MongoClient("mongodb://mongo:27017/"
    "?directConnection=true")["shop"]["orders"]
tf = Path("resume_token.json")
token = json_util.loads(tf.read_text()) if tf.exists() else None


with coll.watch(full_document="updateLookup",
                resume_after=token) as stream:
    for event in stream:  # blocks until a change
        print(event["operationType"],
              event["documentKey"]["_id"])
        tf.write_text(json_util.dumps(stream.resume_token))
```

In mongosh (2nd terminal):

1 insertOne({order_id: "DEMO-1"})

2 updateOne → $set status "paid"

3 Ctrl+C the watcher

4 deleteOne({order_id: "DEMO-1"})

5 Restart → the delete still arrives

The MongoDB Source Connector



- Official, **actively maintained**: latest release **3.1.2** (7 Oct 2026)
- Runs a change stream for you; resume token kept in Connect offsets
- Needs MongoDB 3.6+ (4.0+ for a whole DB) and Kafka Connect 2.1+
- Managed alternative on Atlas: Atlas Stream Processing

```
{ "name": "mongo-orders-source", "config": {
 "connector.class":
  "com.mongodb.kafka.connect.MongoSourceConnector",
 "connection.uri": "mongodb://mongo:27017",
 "database": "shop", "collection": "orders",
 "startup.mode": "copy_existing",
 "change.stream.full.document": "updateLookup",
 "heartbeat.interval.ms": "10000" } }
```

Updates and Deletes in Kafka

PART 8 · WHAT LANDS IN THE TOPIC, PER SETTING

Connector setting	insert	update	delete
Default (full.document = "")	Full event + fullDocument	Deltas only (updateDescription)	documentKey only
full.document = updateLookup	same	Deltas + current fullDocument (may be newer or null)	documentKey only
publish.full.document.only = true	Just the document	Just the current document	Dropped — nothing to publish
… + tombstone.on.delete = true	—	—	Tombstone: key = _id, value = null

Record **key** = documentKey by default → MERGE on _id downstream

"Before" image: full.document.before.change + pre-images (MongoDB 6.0+)

What Happens on Failure

Failure	Connector does	You do
Worker restarts	Resumes from the token in Connect offsets	Nothing — expect a few duplicates (at-least-once)
Down longer than the oplog window	“Invalid resume token” → stops	New offset.partition.name + startup.mode = copy_existing (re-snapshot)
Quiet collection, busy cluster	Token ages out while running	Set heartbeat.interval.ms (default 0 = off)
Bad record	Stops (mongo.errors.tolerance = none)	= all + dead-letter topic, and monitor it

⚠ **Know your oplog window before you get paged.**

Source-Side Checklist

PART 8 · BEFORE YOU CONNECT A MONGODB SOURCE

1. **Oplog window** longer than your longest outage · slides 31, 41
2. **Decide how deletes arrive**: tombstones or they vanish · slide 40
3. **Profile types and shapes**: $type, schema_v, a $jsonSchema contract · slides 15, 16, 26
4. **Idempotent sink**: MERGE by _id, expect duplicates · slides 37, 41
5. **Quiet collections**: set heartbeat.interval.ms · slide 41

Summary and Practice

PART 8 · THREE TAKEAWAYS

MongoDB is the app's **source of truth**; analytics happens downstream

Flexible schema → **drift, mixed types, arrays** — you must detect them

Oplog → **Change Streams** → Kafka connector → lakehouse

Practice · 60 min `docker compose up -d → open lab/handout.md → Setup`

SQL → MongoDB Vocabulary

PART 1 · WHY MONGODB?

SQL	MongoDB
database	database
table	collection
row	document (BSON)
column	field (can be nested)
primary key	_id (ObjectId)

SQL	MongoDB
WHERE	filter { field: value }
SELECT col	projection { col: 1 }
GROUP BY	$group
JOIN	$lookup (or embed)
ALTER TABLE	nothing — no schema

Two rows to remember: **_id** is our downstream MERGE key · **no ALTER TABLE** means changes arrive unannounced